

Cahier des charges

SAÉ 3.02 - Maquette d'un carrefour connecté

Team System

Alessandro BAUER-SICARI et Yacine BENNOUNE

23 septembre 2026

1. Présentation du projet

1.1 Contexte

Le client travaille dans la régulation des déplacements des véhicules d'urgence : pompiers, secours et police. Il souhaite faciliter leur passage dans les carrefours urbains.

Dans le cadre de la SAÉ, notre binôme représente un cabinet de conseil et de développement qui répond à un marché public. Nous devons réaliser une maquette pour montrer que la solution est réalisable. Un entretien complémentaire avec le professeur/client a eu lieu le 15 septembre 2026.

1.2 Problématique

Un véhicule d'urgence peut être bloqué par un feu rouge, une file de voitures ou un carrefour encombré. Lui donner directement le feu vert pourrait aussi mettre les autres usagers en danger.

Comment réduire son temps de traversée tout en maintenant la sécurité des autres usagers ?

1.3 Solution proposée

Nous allons développer une simulation Python d'un seul carrefour. Le véhicule d'urgence se déplacera d'un point A, à l'entrée, vers un point B, à la sortie. Le système adaptera les feux pour faciliter son passage.

Le fonctionnement prévu est le suivant :

1. Le véhicule d'urgence envoie une demande de priorité.
2. Le serveur arrête les flux incompatibles et sécurise le carrefour.
3. Le véhicule traverse lorsque la voie est libre.
4. Les feux reprennent leur cycle normal et le temps de parcours est enregistré.

1.4 Utilisateur et limites

L'utilisateur final de l'application est le **régulateur des véhicules d'urgence**. Il pourra lancer, mettre en pause, réinitialiser et arrêter la simulation. Il réglera la densité du trafic de 0 à 100 %. Une densité de 100 % correspondra à un carrefour saturé ; le nombre de véhicules sera ajusté pendant les essais.

La maquette restera limitée à un carrefour simulé. Elle ne commandera aucun feu réel et ne sera pas reliée à de vrais véhicules.

2. Spécifications techniques

2.1 Langage et environnement

Nous retenons **Python 3.12** (compatibilité à vérifier lors de la recette), **PyCharm** et un environnement virtuel **venv**. Le fichier **requirements.txt** indiquera les versions exactes des dépendances.

Le code sera suivi avec Git et publié sur un dépôt GitHub public. La compatibilité visée est Windows et Linux pour les programmes Python, avec MariaDB sur Linux. Les configurations testées seront précisées dans le README.

2.2 Architecture retenue

Un serveur central reliera trois clients sur un ordinateur ou un réseau local. La maquette simulera une communication **V2I (véhicule-infrastructure)** entre le véhicule d'urgence et le carrefour, à l'aide de sockets Python.

Composant	Rôle
Serveur de régulation	Reçoit les informations, décide de la priorité, coordonne les feux et enregistre les résultats.
Client véhicule d'urgence	Envoie son identifiant, sa position, sa direction, sa vitesse et sa demande de priorité.
Client carrefour	Applique les ordres des feux et transmet l'état du carrefour.
Client régulateur	Affiche la simulation et permet de la piloter.

Les usagers ordinaires seront simulés localement, sans connexion réseau individuelle. Dans le scénario de base, ils respecteront les feux et les règles de circulation.

2.3 Outils et bibliothèques

Outil	Utilisation
PyQt6	Interface avec QtWidgets, QtCore et QtGui ; affichage 2D avec QGraphicsScene et QGraphicsView.
socket, json, select	Sockets TCP brutes et messages JSON ; select si nécessaire.
QThread, pyqtSignal, threading	Réseau et traitements sans bloquer l'interface ; synchronisation des tâches.
MariaDB et son connecteur	Stockage des paramètres, des temps mesurés et des résultats. Connecteur autorisé par le professeur.
math, random, time, datetime	Calculs, génération du trafic et mesure du temps.
matplotlib ou pyqtgraph	Graphiques facultatifs si les statistiques sont développées.

2.4 Contraintes à respecter

Les workers ne modifieront jamais directement les widgets : les mises à jour passeront par les signaux et slots Qt. Les feux devront respecter les phases compatibles, un délai de sécurité et l'occupation du carrefour avant tout passage prioritaire.

Interdits : Tkinter, CustomTkinter, Pygame, Kivy, wxPython, Twisted, Scapy et MQTT/paho-mqtt. Le format précis des messages, les ports réseau et le schéma de la base restent à définir.

3. Périmètre fonctionnel et coût humain

MoSCoW : Must = indispensable ; Should = à faire si le temps le permet ; Could = bonus ; Won't = hors périmètre. La complexité va de 1 (simple) à 5 (difficile).

Les charges ci-dessous sont des **heures de travail cumulées pour le binôme**, et non des heures par étudiant.

Fonctionnalité / tâche	Priorité	Complexité sur 5	Charge binôme
Cadrage et CdC : séances des 9 et 10/09	Must	2	14 h
Serveur central et trois clients	Must	4	7 h
Sockets TCP et protocole JSON	Must	4	6 h
Interface PyQt6 et commandes de simulation	Must	4	9 h
Affichage du carrefour et des feux	Must	3	6 h
Simulation des voitures classiques	Must	3	5 h
Véhicule d'urgence de A à B	Must	3	5 h
Piétons, vélos et bus	Should	4	4 h
Threads et signaux Qt	Must	4	5 h
Détection de la demande de priorité	Must	3	4 h
Algorithme de priorité et transitions des feux	Must	5	8 h
Prévention des collisions et phases incompatibles	Must	5	5 h
Densité réglable de 0 à 100 %	Must	3	4 h
Mesure du temps A-B et du temps d'attente	Must	2	3 h
Enregistrement des résultats dans MariaDB	Must	4	5 h
Tableau de bord des indicateurs	Should	3	3 h
Historique des simulations	Should	3	3 h
Comparaison avec et sans priorité	Should	3	4 h
Contrôle manuel avancé des feux	Could	3	2 h
Journal des événements et alertes	Should	3	2 h
Tests, corrections et recette	Must	4	6 h
Documentation finale, installation et portfolios	Must	3	4 h
Ville ou plusieurs carrefours	Won't	5	Non prévu
Connexion à de vrais feux ou véhicules	Won't	5	Non prévu
Total si toutes les fonctions sont réalisées			114 h

Indispensable : 96 h. Options : 18 h. Limite : 100 h pour le binôme. Les options seront sélectionnées selon le temps restant ; elles ne pourront pas toutes être réalisées dans cette enveloppe.

3. Périmètre fonctionnel et coût humain

Suite : organisation et estimation des coûts

3.1 Organisation du travail

Le cadrage a été fait ensemble. Les séances des 9 et 10 septembre (13 h 30 à 17 h) représentent 7 h par étudiant, soit **14 h pour le binôme**. Pour la suite, nous répartirons les modules et chacun conservera ses commits et ses preuves personnelles.

La limite reste de **50 h par étudiant, soit 100 h au total**. Le cadrage et les tâches indispensables représentent 96 h estimées : il reste donc au plus 4 h pour les options ou les imprévus. Toute heure de travail supplémentaire déjà effectuée devra être déduite de cette marge. Le suivi hebdomadaire permettra d'ajuster le périmètre.

3.2 Coût de la maquette

100 h × 50 €/h = 5 000 € HT au tarif pédagogique de référence. Ce montant valorise le temps du binôme ; il ne correspond pas à une facture réelle.

3.3 Estimation de la simulation finale

Le client demande aussi une estimation de l'étape suivante : une simulation plus complète, avec davantage de scénarios et des tests plus poussés. Le tableau ci-dessous présente l'estimation centrale actuelle.

Lot de travail	Charge estimée
Architecture et conception détaillée	240 h
Moteur de simulation et scénarios	600 h
Réseau, gestion des pannes et journalisation	320 h
Interface, tableau de bord et historique	320 h
Base de données, statistiques et rapports	220 h
Sécurité, prévention des collisions et tests	420 h
Documentation, installation et recette	280 h
Total central	2 400 h

Fourchette retenue : 1 800 à 3 000 heures. À 50 €/h, cela représente **90 000 à 150 000 € HT**, avec un point central de 120 000 € HT.

À titre indicatif, avec un tarif professionnel de 90 €/h, le coût du travail serait de 162 000 à 270 000 € HT. En ajoutant 10 000 à 25 000 € de frais techniques, le budget serait d'environ **172 000 à 295 000 € HT**.

Ces montants sont des estimations de travail à affiner après la maquette. Ils concernent le logiciel et son environnement de test. Ils n'incluent pas l'installation de carrefours réels, les travaux routiers ni leur maintenance.

4. Planning et livrables

4.1 Rétroplanning

Les dates intermédiaires sont des objectifs de travail. Les jalons permettront de vérifier l'avancement chaque semaine.

Période 2026	Travail prévu	Jalon attendu
9 au 23 septembre	Analyse du besoin, entretien du 15/09 et rédaction du CdC.	CdC prêt à présenter.
23 au 30 septembre	GitHub, environnement, architecture et préparation MariaDB.	Structure du projet prête.
1 au 11 octobre	Interface, carrefour et feux.	Première IHM fonctionnelle.
12 au 23 octobre	Serveur, clients et protocole TCP/JSON.	Échanges réseau testés.
24 au 31 octobre	Trafic, densité et véhicule d'urgence.	Trajet A-B simulé.
1 au 7 novembre	Priorité, sécurité et base de données.	Priorité et sauvegarde testées.
8 au 11 novembre	Fonctions complémentaires selon le temps restant.	Version bêta au 11/11.
12 au 14 novembre	Recette, corrections et documents finaux.	Version finale prête.

4.2 Livrables et échéances

Livrable	Contenu attendu	Échéance
Lien GitHub public	Code modulaire et documenté ; requirements.txt à la racine ; README : équipe, installation, venv, configuration et ordre de lancement.	15/11/2026 à 23 h 59
Bilan_Projet.pdf	État de chaque fonction (réalisée, partielle ou non réalisée), difficultés, solutions, organisation et écarts de planning. À la racine du dépôt.	Avec le dépôt final
IAGraphie.pdf ou section du bilan	Usages de l'IA, prompts, vérifications, analyse critique et réflexion éthique.	Avec le dépôt final
Relevés de décisions	Équipe, dates, horaires, type de séance, tâches, décisions et contributions de chacun.	Chaque semaine
E-portfolio individuel	Contexte, AC du référentiel R&T, analyse réflexive et preuves personnelles.	15/11/2026 à 23 h 59
Auto-évaluation et évaluation du binôme	Bilan de la contribution de chacun, justifié par le travail réalisé.	Du 15 au 30/11/2026 Fin à 23 h 59

4.3 Vérifications avant le rendu

La recette vérifiera le trajet A-B, l'effet de la densité, les échanges réseau, la fluidité de l'IHM, les transitions sûres des feux et l'enregistrement dans MariaDB. Une installation en suivant le README devra permettre de lancer la maquette.

Les fonctions optionnelles seront annoncées comme réalisées uniquement si elles ont été testées.